The Dining Philosophers vs. The Cigarette Smokers

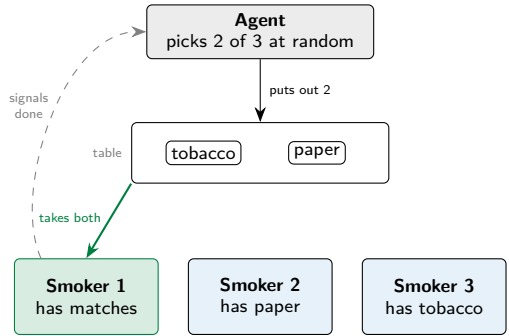
CPE333 Operating Systems – Problem Session 10

Guntee Doungmanee (ID: 67070501003)
Drarun Yookphan (ID: 67070501016)
Rattathamnoon Buranapattana (ID: 67070501037)
Supawit Marayat (ID: 67070501045)

King Mongkut's University of Technology Thonburi

The Cigarette Smokers problem

- Presented by Suhas Patil in 1971 [1, 3].
- Four threads: one **agent**, three **smokers**.
- Ingredients: tobacco, paper, matches. Each smoker has an infinite supply of **one** kind. The agent has an infinite supply of all three [3].
- The agent picks **two different** ingredients at random and puts them on the table.
- The smoker with the **third** ingredient takes both, makes and smokes a cigarette, then signals the agent. The agent puts out two new ingredients.



Premise [3]

The agent is an **operating system** that allocates resources. The smokers are **applications** that need them. Wake an application only when it can proceed, and always wake it when it can.

Why it is hard: the natural solution deadlocks

Patil's restrictions [3]

1. Do not change the agent code. The agent is the OS.
2. No conditional statements and no array of semaphores.

With both, Patil claimed semaphores cannot solve it. Parnas called restriction 2 artificial [2, 3].

The natural solution [3]

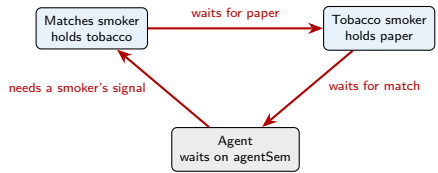
Agent A runs `agentSem.wait()`, then `tobacco.signal()` and `paper.signal()`. Agents B and C are similar. The smokers:

```
# Smoker with matches
tobacco.wait()
paper.wait()
agentSem.signal()
```

```
# Smoker with tobacco
paper.wait()
match.wait()
agentSem.signal()
```

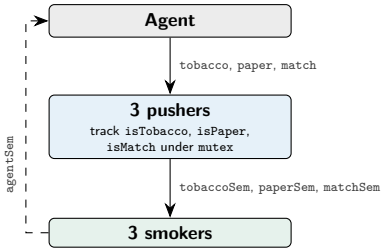
One bad schedule

1. The agent puts out tobacco and paper.
2. The matches smoker takes tobacco. The tobacco smoker takes paper.
3. Each smoker now waits for something it cannot get.



The wait cycle includes the agent: **deadlock** [3].

The solution: helper threads called pushers



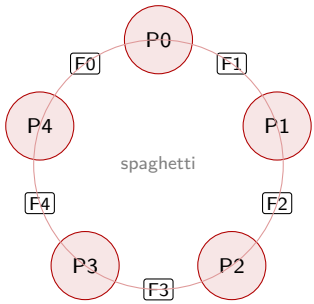
- A smoker waits only on its own semaphore. The pushers signal it only when **both** ingredients are on the table. No smoker holds a partial set.
- Parnas indexes an array of semaphores with an integer. This avoids conditionals [2, 3].

```
# Pusher A: runs when the agent puts out tobacco
tobacco.wait()
mutex.wait()
if isPaper: # paper is already there
    isPaper = False
    matchSem.signal() # wake smoker with matches
elif isMatch:
    isMatch = False
    paperSem.signal() # wake smoker with paper
else:
    isTobacco = True # other item not here yet
mutex.signal()

# Smoker with tobacco
tobaccoSem.wait()
makeCigarette()
agentSem.signal()
smoke()
```

Pushers B and C are similar. Code from [3].

Recap: the Dining Philosophers problem



Philosopher i needs forks i and $(i+1)$ mod 5.

- Proposed by Dijkstra in 1965 [3]. A published form is in [4].
- Five philosophers loop: think, get forks, eat, put forks. Eating needs **two** forks.
- Required properties [3]:
  - one holder per fork;
  - no deadlock and no starvation;
  - more than one philosopher can eat at once.
- **Deadlock:** each philosopher takes the first fork. All five hold one fork and wait for the next. This is a **circular wait**.
- Known fixes [3, 5]:
  - allow only 4 at the table (`footman`);
  - asymmetry: at least one philosopher takes forks in the opposite order;
  - Tanenbaum: a state array plus one semaphore each.

Difference: Cigarette Smokers vs. Dining Philosophers

	Dining Philosophers	Cigarette Smokers
Origin	Dijkstra, 1965 [3]. Paper in 1971 [4]	Patil, 1971 [1, 3]
Threads	Five identical philosophers (peers)	One agent and three smokers with different roles
Shared items	Five forks. Each fork sits between two neighbors and is put back after eating	Ingredients that the agent puts on the table. One smoker picks up both
What a thread needs	Its two neighboring forks. Fixed by its seat	The two ingredients it lacks. Changes each round with the agent's random choice
Who may proceed	Any philosopher whose two forks are free. Several can eat at once	Exactly one smoker per round. The agent waits for its signal
Cause of deadlock	Circular wait: each philosopher holds one fork and waits for the next	Smokers take one needed item each, then wait. The wait cycle includes the agent
Typical fix	Limit the seats, break the symmetry, or use a state array [3, 5]	Add helper threads (pushers) that track the table and wake the right smoker [2, 3]
Extra constraint	None beyond deadlock freedom, starvation freedom, and concurrency	Agent code stays unchanged [3]
Models	Peers that compete for shared resources	An OS that allocates resource combinations to applications [3]

What the differences mean

Deadlock view. Deadlock needs four conditions: mutual exclusion, hold and wait, no preemption, and circular wait [6].

- **Philosophers:** the circular wait is among the peers. A fix removes the cycle (asymmetry) or guarantees a free fork (4 seats).
- **Smokers:** the problem is hold and wait on a *partial* set of ingredients. Pushers remove it: a smoker is woken only when it can get both.

Three main differences

1. **Peers vs. a driver.** Philosophers compete with each other. In the smokers problem the agent decides who can run.
2. **Fixed vs. changing needs.** A philosopher always needs the same two forks. A smoker's needs depend on the round.
3. **Restrict vs. coordinate.** Philosopher fixes limit or order the requests. The smokers fix add a thread that holds the state and decides.

Takeaway

Taking resources one at a time can deadlock in both problems. Dining Philosophers asks how peers share fixed resources safely. Cigarette Smokers asks how to match changing resource sets to the one thread that can use them. Patil's restrictions decide whether semaphores are enough [3].

References

[1] S. Patil, "Limitations and Capabilities of Dijkstra's Semaphore Primitives for Coordination among Processes," Computation Structures Group Memo 57, Project MAC, MIT, Feb. 1971. https://wiki.eecs.yorku.ca/course_archive/2014-15/W/6490A/_media/public:patil.pdf

[2] D. L. Parnas, "On a solution to the cigarette smokers' problem (without conditional statements)," *Communications of the ACM*, vol. 18, no. 3, pp. 181–183, Mar. 1975. https://doi.org/10.1145/360680.360709

[3] A. B. Downey, *The Little Book of Semaphores*, 2nd ed., version 2.2.2, Green Tea Press, 2016. Sections 4.5 and 4.6. https://greenteapress.com/wp/semaphores/

[4] E. W. Dijkstra, "Hierarchical ordering of sequential processes," *Acta Informatica*, vol. 1, no. 2, pp. 115–138, 1971. https://doi.org/10.1007/BF00289519

[5] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2015. Section 2.5.1, "The Dining Philosophers Problem."

[6] E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, Jun. 1971. https://doi.org/10.1145/356586.356588